

Structure-Aware SpGEMM on GPUs: A Portable Hash Engine and a Zero-Profiling Dispatcher

APPT 2026 Student GPU Programming Challenge — Technical Report

Target platform: a single NVIDIA A100 80 GB. Development and all measurements reported here were performed on an NVIDIA H100 with our kernels compiled strictly for `sm_80` (no Hopper-only features), so the binaries and the reported behavior port unchanged to the A100; native A100 timings are expected to hold or improve (see §3).

Abstract

We study general sparse matrix–matrix multiplication (SpGEMM) for the two products of the challenge — the self-product $C = AA$ and the transpose (Gram) product $C = AA^\top$ — over real matrices from the SuiteSparse collection. Real matrices are structurally heterogeneous, and our central result is that **no single implementation wins across them**: our custom hash-based engine beats NVIDIA cuSPARSE by a geometric mean of $2.24\times$ on AA over the small-matrix regime, yet it is slower in aggregate on a few large, highly irregular matrices, while cuSPARSE itself suffers pathological slowdowns (up to $190\times$) on structured stiffness matrices. We therefore contribute (1) a **portable hash-based SpGEMM engine** — per-row-sized shared-memory hashing with a global-memory fallback, unified across both products via an explicit transpose, plus a symmetry-exploiting variant for $AA^\top$ — that is byte-exact against cuSPARSE on all 258 evaluated tasks; and (2) a **zero-profiling, structure-only dispatcher** that selects the faster of {ours, cuSPARSE} per matrix from cheap structural descriptors. At runtime the dispatcher reaches $2.63\times$ **over cuSPARSE**, capturing **88% of the oracle** selection gain, and its dominant decision feature is the **row-length imbalance** (coefficient of variation of nonzeros per row) — directly reflecting the irregular-workload nature of the problem.

1. Introduction

SpGEMM is a core kernel in scientific computing, graph analytics, and modern AI systems. On GPUs its performance is dominated not by arithmetic but by **irregularity**: unknown output sizes, highly variable row lengths, and data-dependent memory access. The APPT 2026 Student GPU Programming Challenge poses SpGEMM on 100 real SuiteSparse square matrices for two products, AA and $AA^\top$, judged on correctness, performance, robustness, generality, and reproducibility on a single A100 80 GB.

Our central finding is that **generality is the hard part**. Because real matrices span banded PDE operators, power-law graphs, symmetric stiffness matrices, and unsymmetric circuits, a kernel tuned for one class loses on another, and even a strong vendor library has large, structure-dependent performance swings. We turn this into an opportunity: instead of chasing a single universally-fastest ker-

nel, we build a competitive portable engine and pair it with a **cheap structural dispatcher** that routes each matrix to the implementation that solves it fastest.

2. Background and Problem

For A in CSR, the self-product follows Gustavson's row-wise formulation: row i of $C = AA$ is

$$C_{i,:} = \sum_{k \in \text{row}_i(A)} A_{ik} \cdot \text{row}_k(A),$$

and the union of the referenced rows' column sets forms the output pattern. The transpose product $C = AA^\top$ is the Gram matrix

$$C_{ij} = \langle \text{row}_i(A), \text{row}_j(A) \rangle = \sum_k A_{ik} A_{jk},$$

which is **symmetric** and equals $A(A^\top)$ — i.e. an ordinary SpGEMM whose second operand is the transpose of A . The two products therefore stress different access patterns, and $AA^\top$ additionally admits a symmetry optimization (compute only the upper triangle).

Benchmark set. The official test matrices match the *size regime* of the first 100 matrices in SuiteSparse's default ordering (the Harwell–Boeing set; $n \in [24, 44609]$, median $n \approx 800$). We evaluate two groups:

- **Group 1 (official-proxy):** the first 100 square real matrices in default order — the small-to-medium regime of the real test.
- **Group 2 (diversity):** 41 structurally diverse matrices we selected to span size (up to 1.1M rows / 6M nonzeros), pattern symmetry (0.0–1.0), and structure class (PDE/CFD, circuits, power-law graphs, chemistry, quantum), to stress *generality*.

3. Methodology

- **Hardware / build.** Target A100 (`sm_80`). All kernels are compiled `-arch=sm_80` with no Hopper-only features (TMA, wgmma, warp-specialization), so the binaries run identically on the A100. Measurements here are on an H100; because cuSPARSE uses its native (Hopper-capable) library paths while our code is `sm_80`-restricted, the reported relative speedups are, if anything, **conservative** for the A100.
- **Precision.** Double precision (fp64) throughout.
- **Timing.** Full end-to-end SpGEMM per call — structure analysis, symbolic, numeric, and canonical-CSR sort (for $AA^\top$, including the transpose) — median of repeated runs with warm allocations. This matches how cuSPARSE is timed (`workEstimation` + `compute` + `copy` , with the transpose via `csr2csc`).
- **Correctness.** Two criteria per (matrix, task): (i) the output nnz equals cuSPARSE's under **structural SpGEMM semantics**, and (ii) order-independent value checksums ($\sum v$, $\sum |v|$, $\sum v^2$) match a scipy reference within 10^{-6} relative. We observed and documented the **structural-zero convention**: some (i, j) are structurally nonzero but numerically cancel to exactly 0 (e.g. in bc-sstk stiffness matrices); cuSPARSE and our engine keep them (structural), scipy prunes them (numeric), but the value checksums are identical either way.

- **Memory guard.** For $AA^\top$ on power-law graphs the Gram output can be near-dense (the upper bound $\sum_k \text{colcount}(k)^2$ blows up); such matrices are excluded from the development sweep (12 of 41 in Group 2).

4. Custom Kernel Design

A single templated engine computes $C = AB$, instantiated as $B = A$ for the self-product and $B = A^\top$ for the transpose product. Its ingredients:

- **Hashing accumulator, one warp per output row.** Each row accumulates its products into an open-addressing hash table (linear probing, multiplicative hash, robust pure-atomicCAS insert).
- **Per-row hash sizing with two-tier placement.** The table size is $\text{next_pow2}(1.33 \cdot \text{flops}_i)$, where $\text{flops}_i = \sum_{k \in \text{row}_i(A)} \deg_B(k)$ upper-bounds the output row's nnz. If it fits the shared-memory budget ($\text{SH_CAP} = 1024$ entries) the row hashes in **shared memory** (fast intra-warp atomics); otherwise it spills to a per-row region of a **global-memory arena**. This realizes the row-binning idea of nsparse/spECK within a single kernel.
- **Two-phase symbolic + numeric.** A symbolic pass counts distinct columns to size C ; a numeric pass accumulates values, then a warp-uniform compaction emits (row $\cdot n + \text{col}$, value) pairs.
- **Canonical CSR via CUB radix sort** of the 64-bit row $\cdot n + \text{col}$ keys, with sort temporary storage allocated once and reused across calls.
- **Transpose product.** $AA^\top$ materializes $A^\top$ with an atomic counting-sort transpose (timed as part of the operation) and runs the same engine.
- **Symmetric variant ($AA^\top S$).** Exploiting that $C = AA^\top$ is symmetric, we hash only $j \geq i$ (halving the accumulation work) and mirror the upper triangle to the full matrix. It is byte-exact against the plain $AA^\top$ on all 129 evaluated matrices; its payoff is **size-dependent** (Table 2) — net-negative on small matrices (mirror + full-result sort outweigh the halved hashing) but $1.12\text{--}1.26\times$ on medium/large — so the symmetry optimization is itself a candidate for dispatch.

5. Correctness

Across **all 258 evaluated (matrix, task) pairs we observe 0 mismatches** against cuSPARSE, on both structural nnz and value checksums. Reaching this exposed a subtle, instructive bug: a shared-memory hash whose computed capacity could exceed SH_CAP and **overflow into the neighbouring warp's shared slice**, producing over-counted nnz and nondeterminism. We localized it by (a) forcing every row down the global path, which stayed correct, and (b) noticing that for a *symmetric* matrix AA and $AA^\top$ disagreed by a single entry — impossible if correct. The fix ties the shared/global choice to whether the computed capacity actually fits SH_CAP , and hardens the insert to pure atomicCAS. We regard this correctness discipline — structural-zero semantics, symmetry as a self-check, and byte-exact cross-validation — as part of the contribution.

6. Performance

Speedups are cuSPARSE time $\div$ our time; a value > 1 means we are faster. Table 1 reports per-group distributions for both products.

Table 1 — Per-group speedup of our engine vs cuSPARSE (fp64).

Group	Product	geomean	median	wins ($\geq 1\times$)	range
G1 (official-proxy, 100)	AA	2.24$\times$	2.00 $\times$	82/100	0.04–190 $\times$
G1	$AA^\top$	1.68 $\times$	1.46 $\times$	78/100	0.03–91 $\times$
G2 (diverse, 29 ran)	AA	1.82 $\times$	1.94 $\times$	22/29	0.02–61 $\times$
G2	$AA^\top$	0.94 $\times$	1.72 $\times$	18/29	0.05–17 $\times$

Three observations frame the rest of the report:

- On the **official small-matrix regime (G1)**, our engine solves the whole suite in **387 ms vs cuSPARSE's 1029 ms** — **2.66 $\times$ faster in aggregate**, winning on 78–82% of matrices.
- cuSPARSE is **non-robust**: there are 37 G1 cases where we are more than $5\times$ faster, topped by stiffness matrices (bcsstk10 AA **190 $\times$** , bcsstk23 $104\times$, bcsstk06 $96\times$) where cuSPARSE hits a pathological path.
- We are **explicit about the failure mode**: on Group 2's large, highly irregular matrices we lose in aggregate ($0.71\times$), driven by a few outliers where our single hashing strategy is far slower (e.g. shermanACb $AA \approx 50\times$ slower). This is exactly where a dispatcher must fall back to cuSPARSE — motivating §7.

The symmetric variant of $AA^\top$ (§4) is a second axis of the same lesson: its benefit depends on matrix size. Table 2 breaks the $AA^\top S$ -vs-plain speedup down by output size.

Table 2 — Symmetric $AA^\top$ ablation ($AA^\top S$ vs plain $AA^\top$; 129 matrices, 0 mismatches). Speedup is plain time $\div$ symmetric time.

size (nnz)	matrices	geomean speedup
small ($< 10^5$)	104	0.92 $\times$ (slower)
medium (10^5 – 10^6)	16	1.12 $\times$
large ($> 10^6$)	9	1.26 $\times$
all	129	0.97 $\times$

The optimization halves the accumulation work but adds a mirror pass and sorts the full ($2\times$) result. On the small-matrix regime that dominates the suite the overhead wins; on larger $AA^\top$ it pays off — a clean example of a kernel-level optimization whose benefit is itself structure/size dependent, and thus a further argument for dispatching.

7. Structure-Aware Dispatch (Zero Profiling)

Since neither our engine nor cuSPARSE dominates, we select between them **per (matrix, task) from cheap structural descriptors** — matrix size n , nnz, mean degree, degree coefficient-of-variation deg_cv , an estimated fill factor, and pattern symmetry — computed with a single pass over the CSR, with no timing or trial runs at decision time. A shallow decision tree over these features reduces to the interpretable rule

choose ours $\iff (\text{deg_cv} \leq 0.63 \wedge \text{fill} \leq 53) \vee (\text{deg_cv} > 0.63 \wedge \text{nnz} > 20\text{k} \wedge \text{fill} \leq 47)$.

Its **dominant feature is** `deg_cv` (**row-length imbalance**), with importance 0.81 against 0.12 (`fill`) and 0.07 (`nnz`): the decision is driven by exactly the irregularity that makes SpGEMM hard.

We implemented this dispatcher **online** (`bench/dispatch.cu`): at call time it computes the descriptors, applies the rule, and runs the chosen implementation. Table 3 reports the realized whole-suite result, together with the always-one-implementation baselines, the offline (measured-time) analysis, and the oracle upper bound.

Table 3 — Whole-benchmark dispatch (258 tasks, total wall time).

Policy	total	vs cuSPARSE	notes
cuSPARSE-always	1520 ms	1.00×	vendor baseline
ours-always	1083 ms	1.40×	strong, but keeps the outliers
structure-only dispatcher (offline)	628 ms	2.42×	92% pick accuracy, 83% of oracle
online dispatcher (real runtime)	579 ms	2.63×	picks ours 211 / cuSPARSE 47, 88% of oracle
oracle (min per task)	446 ms	3.41×	upper bound

The online dispatcher realizes $2.63\times$ **over cuSPARSE and recovers 88% of the oracle gain** — on par with the idealized offline rule ($2.42\times$; the two agree within cuSPARSE's timing noise). The decision itself is essentially free: the descriptor overhead is 0.01–0.14 ms per matrix ($< 1\%$) for the small-matrix regime, and 8.6% aggregated over the whole suite — dominated by the host $O(\text{nnz})$ pass on the largest matrices, and straightforwardly reducible by computing the descriptors on the GPU or fusing them into the CSR-load pass. This validates the thesis end-to-end: a **zero-profiling, structure-only decision recovers most of the oracle selection gain at near-zero cost**, and it does so by keying on the row-length imbalance that defines the workload's irregularity.

8. Related Work

Hash-based SpGEMM (nsparse), lightweight-analysis strategy selection (spECK), high-shared-utilization accumulators (HSMU-SpGEMM), merge-based accumulation (bhSPARSE), and learned output-size prediction all inform our engine. Our positioning is twofold: (i) a **portable sm_80 engine** that unifies AA and $AA^\top$ with a symmetric specialization; and (ii) a **zero-training, structure-only dispatcher over {custom, vendor}**. The latter is closest in spirit to spECK's per-matrix analysis, but it selects *between implementations* — including the vendor library — rather than only among a single kernel's internal variants.

9. Limitations and Future Work

- **Large, highly irregular matrices.** Our single hashing strategy is not tuned for very large / high-fill rows; a merge-based accumulator branch and finer intra-matrix (per-row-block) dispatch are the clear next steps.
- **H100 measurements, `sm_80` code.** Numbers should be reproduced natively on the A100; the relative gains are expected to hold or improve as cuSPARSE loses its Hopper-path advantage.
- **Heuristic vs learned dispatcher.** The shallow tree is a stand-in for a principled, trained cost model over structural descriptors — a natural path to a stronger, more generalizable dispatcher.
- **Benchmark proxy.** Group 1 proxies the official set by size regime; the exact official matrices may differ in content.

10. Conclusion

Real-world SpGEMM is fundamentally a generality problem. We built a correct (0/258 mismatches), portable, competitive hash engine for both challenge products, showed that neither it nor the vendor library wins everywhere, and turned that fact into a **profiling-free, structure-aware dispatcher that reaches $2.63\times$ over cuSPARSE (88% of the oracle), driven by row-length imbalance**. The approach is robust by construction: it routes each matrix to whatever solves it fastest — which is precisely what a general, real-world SpGEMM solver needs.

Reproducibility. Code and drivers: `src/` (kernels), `bench/` (cuSPARSE baseline, sweeps, dispatcher), `data/` (manifests, structural descriptors). Build with `-arch=sm_80`; matrices are fetched from SuiteSparse via `src/fetch_and_prep.py`. All numbers are self-measured on the current machine.